

Make nav tab clicks feel instant

Context

DeckGenie (Next.js 16 App Router, React 19) has a sidebar/mobile-drawer tab bar (Home, Practice, Vocabulary, Review, Progress, Settings). Clicking a tab currently feels laggy. Root cause, confirmed by reading the code: **there is no shared layout** — `src/app/layout.tsx` is just the bare root (fonts + theme script). Every authenticated page (`dashboard`, `practice`, `vocabulary`, `review`, `progress`, `settings`) individually wraps its own content in `<AppShell>` (which renders `Sidebar` + `MobileTopBar` + `MobileDrawer` + `RightRail`). Because the shell lives inside each page rather than a shared layout, **every tab click unmounts the entire previous page's tree — including the sidebar — and mounts a fresh one.**

That remount is the actual latency source:

- `Sidebar.tsx` calls `useProgress()` and `useUserEmail()` itself. `useProgress()` (`src/hooks/useProgress.ts`) fires `supabase.auth.getSession()` + 3 parallel Supabase queries in a mount effect — this full round trip re-runs on **every single click**, stacked on top of whatever the destination page's own hooks also fetch (there's no React Query/SWR/shared cache anywhere, just per-hook localStorage caching).
- The nav active-pill FLIP animation (`NavLink.tsx`, `motion.span layoutId="nav-active-pill"`) can't actually slide between tabs, because Framer Motion's `layoutId` matching needs both instances alive in the same tree across the transition — right now it just pops in fresh.
- Every mount effect in `Sidebar` (dropdown listeners, settings-change listener) re-runs, plus a full new tree for React to reconcile, before the destination page's own content can settle.

The fix: hoist `AppShell` into a shared Next.js layout (via a route group) so the shell stays mounted across tab clicks and only the page content swaps. This also incidentally makes the pill animation work correctly for the first time. Two secondary latency contributors get small targeted fixes alongside it: duplicate `useProgress()` instances beyond just `Sidebar`, and un-debounced/uncancelled fetches on the vocabulary filter pills.

Wrinkle: settings onboarding

`src/app/settings/page.tsx` conditionally skips `<AppShell>` when `?firstRun=true` (onboarding), so a brand-new user can't escape via the sidebar's nav/sign-out before their settings are saved. A layout can't read `searchParams` and a page can't unwrap a parent layout, so this logic has to move from "the page decides whether to wrap itself" to "the page tells the (now-persistent) shell to hide its chrome."

Implementation

1. Route group + shared layout

Move the six shell-using routes into a route group (route groups don't affect URLs):

```
src/app/dashboard → src/app/(app)/dashboard
src/app/practice → src/app/(app)/practice
src/app/progress → src/app/(app)/progress
src/app/review → src/app/(app)/review
src/app/settings → src/app/(app)/settings
src/app/vocabulary → src/app/(app)/vocabulary
```

`login/`, `signup/`, `root page.tsx`, `auth/callback/`, `api/` stay put — untouched.

New `src/app/(app)/layout.tsx`:

```
import { AppShell } from '@components/shell/AppShell'
import { ProgressProvider } from '@hooks/useProgress'

export default function AppLayout({ children }: { children: React.ReactNode }) {
  return (
    <ProgressProvider>
      <AppShell>{children}</AppShell>
    </ProgressProvider>
  )
}
```

2. Right-rail content: context-based slot (replaces the `rightRail prop`)

`AppShell` currently takes a `rightRail` prop because pages render it directly. Once `AppShell` is in the layout, pages are no longer its direct caller — and `practice/page.tsx` needs to switch its rail (`PracticeRightRail` vs `AnalysisRightRail`) based on internal `analysisMode` state, which a route-based parallel slot can't express. Use a small context slot instead.

New `src/components/shell/AppShellSlots.tsx`:

- `AppShellSlotsContext` ({ `setRailContent`, `setChromeHidden` }).
- `useRightRail(node: ReactNode)` — publishes rail content via `useLayoutEffect` (sets on mount, clears on unmount) so navigating to a page with no rail always clears any previous page's leftover content, and so the swap lands in the same paint as the route change.
- `useHideAppShellChrome(hidden: boolean)` — same pattern, used only by settings onboarding.

Edit `src/components/shell/AppShell.tsx`:

- Drop the `rightRail` prop; own `railContent/chromeHidden` state instead.
- Provide `AppShellSlotsContext`.
- When `chromeHidden`, render bare `{children}` (wrapped in the context provider so the hook call itself doesn't throw).
- Otherwise render the existing tree, using `railContent` where `rightRail` used to be.

- Keep the existing `drawerOpen/prevPathname` close-on-route-change logic as is — it was previously inert (since `AppShell` remounted before a drawer could ever be left open across a nav) and becomes correctly active once `AppShell` stops remounting.

3. Per-page edits (drop the `<AppShell>` wrap)

- `dashboard/page.tsx` — add `useRightRail(<DashboardRightRail />)` once, near the top; strip `<AppShell rightRail={...}>...</AppShell>` in both the loading and loaded return blocks down to the inner `<div>`; remove the `AppShell` import.
- `practice/page.tsx` — add one `useRightRail(...)` call computing `analysisMode && state.sentence ? <AnalysisRightRail ... /> : <PracticeRightRail />`; strip the `<AppShell rightRail={...}>` wrap from both the analysis-mode return (~line 385) and normal-mode return (~line 559) down to their inner content; remove the `AppShell` import.
- `vocabulary/page.tsx`, `progress/page.tsx`, `review/page.tsx` — unwrap `<AppShell>{...}</AppShell>` to just the inner content (no rail on these pages).
- `settings/page.tsx` — replace the `isFirstRun ? form : <AppShell>{form}</AppShell>` branch with a single `useHideAppShellChrome(isFirstRun)` call at the top of `SettingsInner`, then always return the inner `<div>` directly. Remove the `AppShell` import.

4. Collapse duplicate `useProgress()` instances

`Sidebar`, `dashboard/page.tsx`, `practice/page.tsx`, `vocabulary/page.tsx`, `progress/page.tsx`, and indirectly `DailyGoalCard/StreakCard/DashboardRightRail` (via `useTodayStats`) all call `useProgress()` independently — each firing its own `getSession()` + 3-query fetch. Step 1 already kills the worst of it (`Sidebar`'s instance no longer remounts per click), but multiple instances can still fire near-simultaneously on one page load.

Edit `src/hooks/useProgress.ts`: rename the existing hook body to a private `useProgressState()`, then add:

```
const ProgressContext = createContext<ReturnType<typeof useProgressState> | null>(null)

export function ProgressProvider({ children }: { children: React.ReactNode }) {
  const value = useProgressState()
  return <ProgressContext.Provider value={value}>{children}</ProgressContext.Provider>
}

export function useProgress() {
  const ctx = useContext(ProgressContext)
  if (!ctx) throw new Error('useProgress must be used within ProgressProvider')
  return ctx
}
```

No call sites change — the returned shape is identical, so every existing consumer keeps working, now sharing one instance (and one `reload`) instead of each mounting its own. Only `(app)/layout.tsx` needs the new `<ProgressProvider>` wrap from step 1.

5. `useVocabSheet.ts` *filter-pill fix (independent, smaller)*

In `src/hooks/useVocabSheet.ts`:

- Replace `supabase.auth.getUser()` with the cached `supabase.auth.getSession()` (`session?.user ?? null`) in both `fetchWords` and `removeWord` — matches the pattern `useProgress.ts` already uses to avoid a network round trip.
- Add a `requestIdRef` sequence counter in `fetchWords`: increment it at the start of each call, and after the awaited query resolves, bail out (don't call `setWords/setLoading(false)`) if `requestId !== requestIdRef.current` — so rapid filter-pill clicks can't have an older, slower response overwrite a newer one.

Verification

- 1 `npm run dev`, sign in, open devtools Network tab filtered to the Supabase host.
- 2 Click between Dashboard / Practice / Vocabulary / Review / Progress / Settings repeatedly — confirm the sidebar/topbar no longer flicker/remount, the active-pill slides between tabs instead of popping, and no new `getSession/progress/settings/vocab_list` requests fire per click beyond the destination page's own data.
- 3 Visit `/settings` from the sidebar (should show the full shell) and separately via the onboarding flow with `?firstRun=true` (should show no sidebar/nav) — confirm both still behave as before.
- 4 Open the mobile drawer (narrow viewport), tap a nav link, confirm it closes instantly.
- 5 In Vocabulary, click HSK/POS filter pills rapidly — confirm the list settles on the last-clicked filter, not a stale intermediate one.
- 6 `npm run build` to confirm the route-group move doesn't break the Next.js build.